

GIT & DISTRIBUTED VERSION CONTROL

A Student's Practical Guide to Modern Workflows and Local State Tracking

1. The Paradigm Shift: Distributed vs. Centralized VCS

In team-based software engineering, managing parallel changes without a central bottleneck is vital for releasing high-quality systems. Traditional version control systems (like Subversion) are **centralized**, meaning they rely on a single server containing the entire change history. If that server goes offline or has a hardware crash, developers cannot commit or review code history. Modern tools use a **Distributed VCS (DVCS)** like Git, where every developer's local machine maintains a full clone of the repository history.

Feature	Centralized VCS (Subversion)	Distributed VCS (Git)
Repository Model	One central repository on a single server.	Every developer maintains a full clone of the repo history locally.
Local Operations	Requires internet access to commit or view logs.	Committing, branching, and logging are completely local & instant.
Data Redundancy	Single point of failure. Server crash means data loss.	No single point of failure; every copy acts as a complete backup.
Branching Cost	Expensive and slow (requires server-side copies).	Virtually instant and lightweight (pointers to commit hashes).

2. The Git Architecture & Lifecycle States

Unlike centralized models, files in a Git repository transition across three distinct local zones before synchronizing with a remote host (like GitHub, GitLab, or Bitbucket). Understanding this state machine is the key to avoiding data loss and masterfully staging your work.

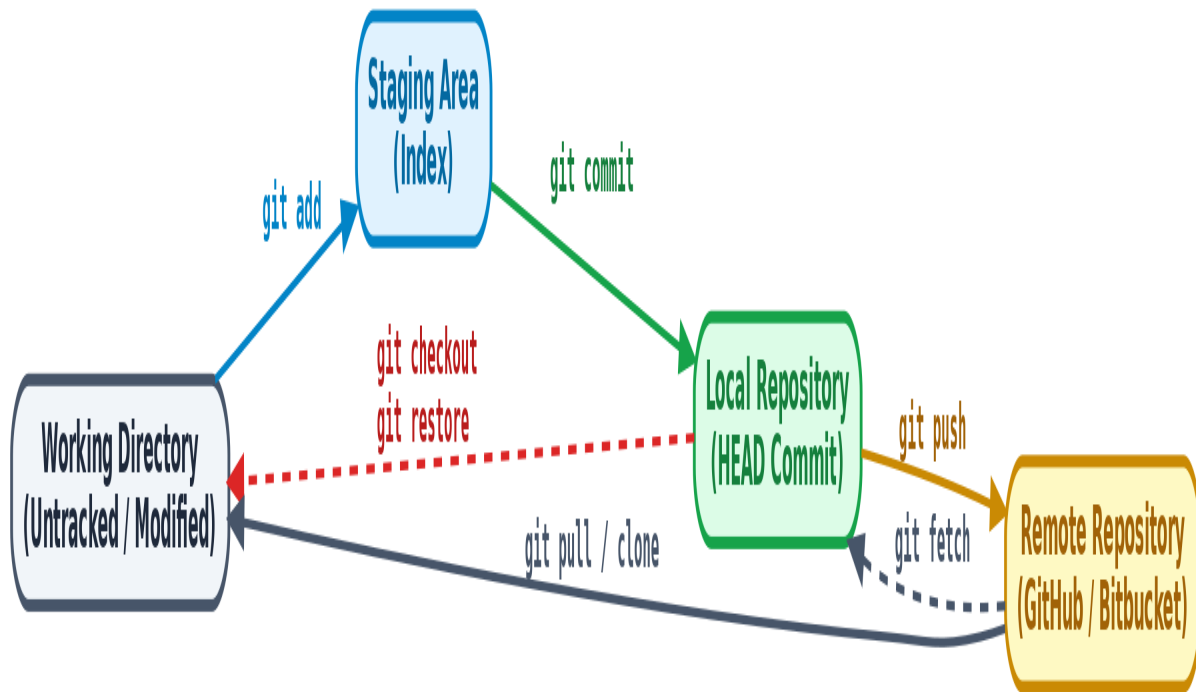


Figure 1.1: Visual layout of file state transitions and associated Git commands.

3. Core Commands: Staging and Tracking Changes

To write and track features, you will operate locally within your shell, moving modified files through staging and into the local database before publishing them to teammates.

```
# Step 1: Initialize a new repository or clone an existing one
$ git init my-awesome-app
Initialized empty Git repository in /workspace/my-awesome-app/.git/

# Step 2: Check the tracking status of files
$ git status
On branch main
Untracked files:
(use "git add <file>..." to include in what will be committed)
main.py

nothing added to commit but untracked files present (use "git add" to track)
```

```
# Step 3: Stage the file (move to Staging Area)
$ git add main.py

$ git status
On branch main
Changes to be committed:
(use "git rm --cached <file>..." to unstage)
new file:   main.py

# Step 4: Commit changes (move to Local Repository) with a descriptive message
$ git commit -m "feat: implement main entrypoint for clinical lookup"
[main (root-commit) d47f5a2] feat: implement main entrypoint for clinical lookup
1 file changed, 15 insertions(+)
create mode 100644 main.py
```

```
# Step 5: Connect and push local history to a remote repository
$ git remote add origin https://github.com/clinic-org/lookup-app.git
$ git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 290 bytes, done.
To https://github.com/clinic-org/lookup-app.git
* [new branch] main -> main
branch 'main' set up to track 'origin/main'.
```

4. Branching, Merging, and Conflict Resolution

Branching isolates developers' daily work to prevent broken code from affecting other teammates. It is extremely cheap in Git, as branches are merely lightweight, movable pointers to specific commits. When a feature is completed and tested, it is integrated back into the main trunk via a merge.

```
# How branches diverge and merge back together:
[C1] <--- [C2] <--- [C3] <--- (main)
\
\---> [C4] <--- [C5] <--- (feature/lookup-fix)

# When merged back, a new '3-Way Merge Commit' integrates both branches:
[C1] <--- [C2] <--- [C3] <----- [C6] <--- (main / HEAD)
\ /
\---> [C4] <--- [C5] <---/
```

The Two Types of Merges:

Merge Type	When It Occurs	What Git Does	History Visual
Fast-Forward	The target branch has NO new commits since the branch diverged.	Moves the branch pointer forward. No new merge commit is made.	Linear line; no intersections.
Three-Way (3-Way)	Both branches have diverged with independent commits.	Finds the common ancestor, merges diffs, and creates a merge commit.	Divergent branches joining at a node.

5. Collaboration Workflows & Team Git Strategies

Software teams must choose a branching strategy that fits their release cycle and team size to prevent integration conflicts. Sommerville's configuration management guidelines stress choosing a process that matches project velocity.

Strategy	Primary Workflow Pattern	Best For	Key Advantage
GitFlow	Highly structured. Uses main, develop, feature, release, and hotfix branches. Workflows are isolated and heavily checked.	Desktop software, formal enterprise releases, and strict compliance environments.	Perfect for formal release schedules; main is always completely stable.
Trunk-Based	Lightweight. Team commits directly to main (or short-lived branches merged multiple times a day). Highly automated.	Cloud-native web systems, continuous delivery, and high-velocity engineering.	Eliminates 'merge hell' by keeping integrations small, fast, and continuous.

6. Troubleshooting Git Disasters (The Student Guide)

As a student or junior engineer, you will inevitably hit Git mistakes. Learn to programmatically recover without erasing your local files.

Disaster 1: Committed your work directly to 'main' instead of a feature branch

```
# Create your new feature branch based on your current main commits
$ git branch feature/patient-lookup

# Roll back 'main' by 1 commit (preserving files in your Working Directory)
$ git reset --soft HEAD~1

# Switch to your feature branch; your staged files will carry over safely
$ git checkout feature/patient-lookup
```

Disaster 2: Accidentally committed a file containing API keys or secrets

```
# Step 1: Remove the file from Git's tracking but KEEP it locally
$ git rm --cached secrets.env

# Step 2: Append the file to .gitignore so it is never committed again
$ echo "secrets.env" >> .gitignore

# Step 3: Amend your commit to erase it from the latest local history
$ git commit --amend -m "refactor: add .gitignore and strip credentials"
# Note: If the secret was pushed, immediately ROTATE the credentials on the server!
```

Disaster 3: Stuck in a 'Detached HEAD' state

```
# Why it happened: You checked out a specific commit hash rather than a branch.
# To recover: Switch back to your local branch immediately.
$ git checkout main
```

Disaster 4: Panicking over Merge Conflicts

```
# When you merge, Git highlights conflicting blocks directly in your code files:
<<<<<< HEAD
return "Appointment confirmed on " + date;
=====
return f"Appointment confirmed on {date} at {time}";
>>>>>> feature/lookup-fix

# Solution: Open your editor, choose the correct lines, delete the conflict markers,
# stage the corrected files, and complete the merge:
$ git add main.py
$ git commit -m "merge: resolve conflicts between main and feature/lookup-fix"
```

TAKEAWAY: THE FORCE-PUSH DANGER ZONE

Never use **git push -f** on shared branches! It completely overwrites your team's historical commits on the server. If you must overwrite your own isolated branch, use **git push --force-with-lease** as a safe alternative, which aborts if someone else has pushed new commits since your last pull.